Fibonicci Series



Sum of numbers is equal to previous two numbers.

PMI, extended form

Base Case :- Same f(0) or f(1)

IH : f(i) is true for all $i \leq R$

IS : Use (2) to prove f(k+1) is true

The above makes our framework a lot stronger as we can assume for everything less than n.

write code for fibonicci number.

explain with first example of single base case.

order and everything ← make sure to properly explain recursion tree ⟸ dry run as well

Make sure to run the dry code & tree after writing the code.

We are doing math & not recursion.

Q: Write a program for a given number n.

1] To print 1 to N.
2] To print N to 1.

mention on how to build a thought process.

make sure to explain the difference properly.

Also explain head vs tail recursion.

Assignment

Q-1 Sum of digits of a number

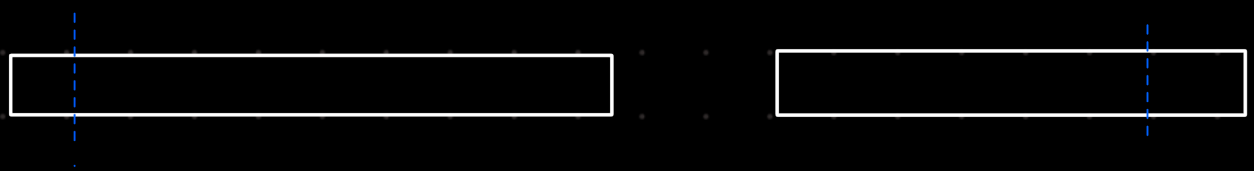
Q-2 Power of a number

Recursion with Arrays

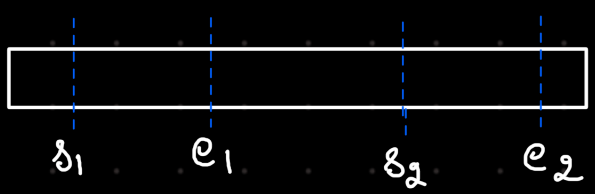
Whenever in arrays, we are dealing with a problem that can be broken & individually be applied to left over part, we can use recursion there.

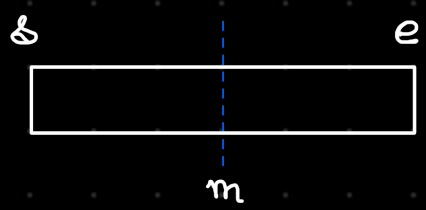
Below are major ways we break our array / list.

1. First or last element removed



2. Using start and end index (eg. binary search)





Q:- Sum of an array using Recursion

Find the sum of an array using recursion, specify the recurrence relation and recursion tree.

Give code solution via head and tail recursion.

Q: Check if array is sorted.

using recursion.

Now, we just have to think on our approach

arr[0] arr[1] how to handle & write code better.

Now, in this explain the two approaches.

Keep on mentioning how understanding should also be clear.

↓

thought process

So it normally doesn't matter if we are doing our work first or not in these kind of problems.

1] Base Case

2] Recursive call

3] Our Calculation.

Q: Given an array find the first, last and all indices of a number.

We have to show multiple views here.

1] First Index
2] Last index
3] All index and passing the list
4] All index but list not getting passed

Explain using id

Can be in software dev practice so explain why we might want to send the list & update same only.

Also make sure to explain doubt which can come.

5] Point → not return nair but rather just call the function.

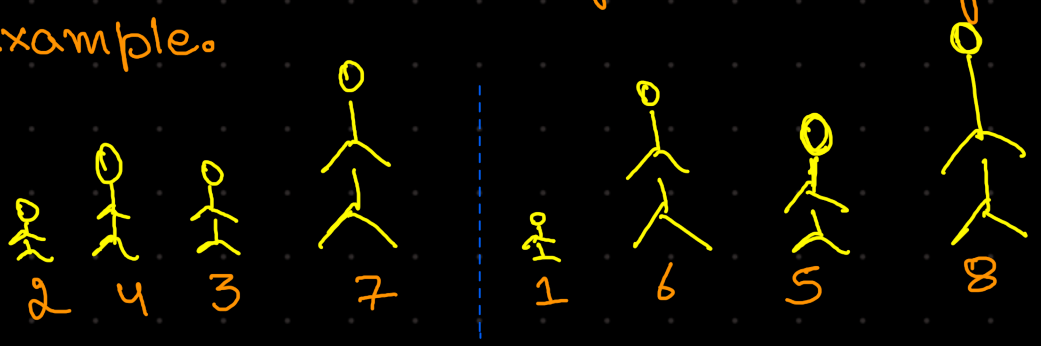
Also brush to create all the diagram and Python Tutor.

Merge Sort

⇒ recursive sorting algorithm

⇒ fast as compared to other sorting (will learn why)

Let's understand merge sort using an example.





first explain intuitively

use hackerank visualizer

list copy as in-place

how will it keep on going still?
even if left over part is sorted.

Quick Sort

⇒ In quick sort, unlike merge sort:

we will first do our work and then let recursion handle the rest.



---

we are now going to portion along last element.

Searching using recursion

→ Linear Search

→ Binary search

Recursion with Strings

→ Just like arrays, for string too we work on them after piching one of the element and then asking recursion to work on rest.

Subsequence

Print vs Return vs return a list/collection.

Helper function way

Print and return subset to R

Tower of Hanoi

Letter Combination of a Keypad

Print and return all codes

Framework to solve any recursion question.

1] Identify if problem can be broken into a smaller same problem.

2] Write recurrence relation.

3] Code

a) Identify and write base cases.

b) Assume (don't Question) the Induction hypothesis.

c]